



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION



Basics of Neural Network & Deep Learning Lab

ENCA354

LAB FILE

BCA sp. in AI&DS (Sec-C)

SUBMITTED BY:

Dhruv
2301201212

SUBMITTED TO:

Anoja Mishra

INDEX

SR. NO.	EXPERIMENTS	PAGE NO.	TEACHER SIGN
01.	Experiment - 1 Neural Network using TensorFlow (Low-Level API)	3-5	
02.	EXPERIMENT 2: Neural Network using Basic Python (From Scratch)	6-8	
03.	EXPERIMENT 3: Neural Network using Keras	9-10	
04.	EXPERIMENT 4: Agriculture Price Prediction (Regression Problem)	11-13	
05.	EXPERIMENT 5: MNIST Digit Classification using ANN	14-16	
06.	EXPERIMENT 6: Plant Disease Detection using CNN	17-19	
07	Conclusion	20-21	

Projects on - Basics of Neural Network and Deep Learning

EXPERIMENT 1: Neural Network using TensorFlow (Low-Level API)

- Build a simple feedforward neural network using TensorFlow 1.x style (placeholders, sessions)
- Understand computation graph and manual weight initialization

Detailed Theory:

A Neural Network is a computational model inspired by biological neural systems. It consists of interconnected neurons organized in layers. Each neuron receives input, processes it using weights and bias, and produces an output using an activation function.

In TensorFlow low-level API (especially 1.x style), computation is performed using a static computation graph. This means we first define the structure of the computation and then execute it using a session.

Key Concepts:

1. Computation Graph: TensorFlow represents operations as nodes and data as edges. The graph defines how data flows and transforms.
2. Placeholders: Placeholders are used to feed input data at runtime. They act as empty variables that will be filled during execution.
3. Variables: Variables store parameters such as weights and biases. These are updated during training.
4. Session: Session is responsible for executing the computation graph.

Working Logic:

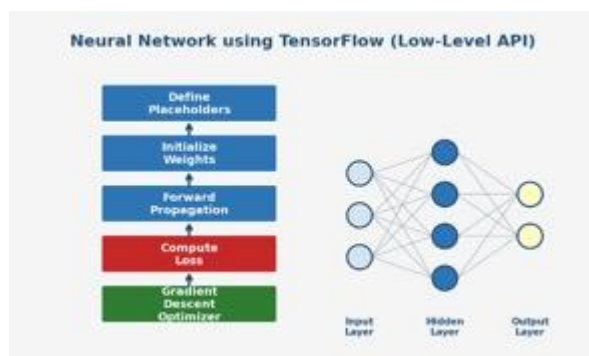
5. Input layer receives data
6. Hidden layer applies transformation using weights and activation
7. Output layer produces prediction
8. Loss is calculated
9. Optimizer updates weights using gradient descent

Activation Functions:

- ReLU: used in hidden layers for non-linearity
- Sigmoid: used in output layer for binary classification

Loss Function: Mean Squared Error is used to measure prediction error.

Result Explanation: The model learns XOR pattern, which is a non-linear problem, demonstrating the power of neural networks.



Code: -

```
import tensorflow as tf
import numpy as np

tf.compat.v1.disable_eager_execution()

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=np.float32)
Y = np.array([[0],[1],[1],[0]], dtype=np.float32)

x = tf.compat.v1.placeholder(tf.float32, [None, 2])
y = tf.compat.v1.placeholder(tf.float32, [None, 1])

W1 = tf.Variable(tf.random.normal([2, 4], stddev=0.5))
b1 = tf.Variable(tf.zeros([4]))
W2 = tf.Variable(tf.random.normal([4, 1], stddev=0.5))
b2 = tf.Variable(tf.zeros([1]))

hidden = tf.nn.relu(tf.matmul(x, W1) + b1)
output = tf.nn.sigmoid(tf.matmul(hidden, W2) + b2)

loss = tf.reduce_mean(tf.square(output - y))
optimizer = tf.compat.v1.train.GradientDescentOptimizer(0.1).minimize(loss)

with tf.compat.v1.Session() as sess:
    sess.run(tf.compat.v1.global_variables_initializer())
    for epoch in range(5000):
        sess.run(optimizer, feed_dict={x: X, y: Y})
        if epoch % 1000 == 0:
            l = sess.run(loss, feed_dict={x: X, y: Y})
            print(f"Epoch {epoch:4d} Loss: {l:.6f}")
    pred = sess.run(output, feed_dict={x: X})
    print("\nPredicted Output:")
    print(pred)
```

Output: -

```
... Epoch    0  Loss: 0.281226
    Epoch 1000  Loss: 0.026759
    Epoch 2000  Loss: 0.007292
    Epoch 3000  Loss: 0.003762
    Epoch 4000  Loss: 0.002446

    Predicted Output:
    [[0.06618906]
     [0.96444315]
     [0.97275674]
     [0.0272745 ]]
```

EXPERIMENT 2: Neural Network using Basic Python (From Scratch)

- Implement a neural network using NumPy only
- Include: Forward propagation, Backpropagation, Gradient descent

Detailed Theory:

This experiment helps in understanding the internal working of neural networks without using any frameworks.

Forward Propagation:

In forward propagation, input data is passed through layers to compute output.

Steps:

- Multiply inputs with weights
- Add bias
- Apply activation function

Activation Function: Sigmoid function is used to map values between 0 and 1.

Error Calculation: Error is calculated as difference between actual and predicted values.

Backpropagation:

Backpropagation is the core learning algorithm of neural networks.

Steps:

10. Compute output error
11. Calculate gradient
12. Propagate error backward
13. Update weights

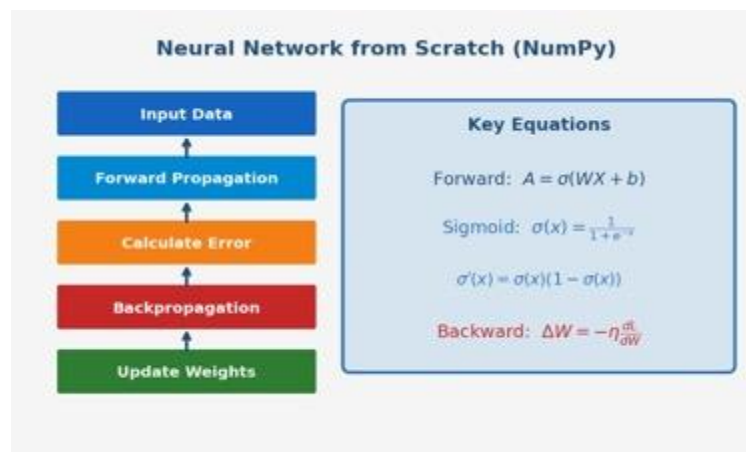
Gradient Descent:

Gradient descent minimizes loss by adjusting weights in the direction of negative gradient.

Important Points:

- Learning rate controls speed of learning
- Too high learning rate causes instability
- Too low learning rate slows training

Result Explanation: The model successfully performs binary classification after multiple iterations.



Code: -

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

def sigmoid_derivative(x):
    return x * (1 - x)

X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([[0],[1],[1],[0]])

np.random.seed(42)
W1 = np.random.randn(2, 4) * np.sqrt(2.0/2)
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1) * np.sqrt(2.0/4)
b2 = np.zeros((1, 1))
lr = 0.5

for epoch in range(10000):

    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)

    error = Y - output
    d_output = error * sigmoid_derivative(output)
    d_hidden = d_output @ W2.T * sigmoid_derivative(hidden)

    W2 += hidden.T @ d_output * lr
    b2 += np.sum(d_output, axis=0, keepdims=True) * lr
    W1 += X.T @ d_hidden * lr
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr

    if epoch % 2500 == 0:
        loss = np.mean(np.square(error))
        print(f"Epoch {epoch:5d} MSE Loss: {loss:.6f}")

print("\nFinal Output:")
print(output)
print(f"Accuracy: {np.mean((output>0.5)==Y)*100:.1f}%")
```

Output: -

```
*** Epoch    0 MSE Loss: 0.268272
    Epoch 2500 MSE Loss: 0.001890
    Epoch 5000 MSE Loss: 0.000675
    Epoch 7500 MSE Loss: 0.000404

    Final Output:
    [[0.01795329]
     [0.98229156]
     [0.98532381]
     [0.01723551]]
    Accuracy: 100.0%
```

EXPERIMENT 3: Neural Network using Keras

- Build a model using Keras Sequential API
- Train on a small dataset (e.g., Iris or dummy data)
- Visualize accuracy and loss

Aim:

To build and train a neural network using Keras Sequential API.

Detailed Theory:

Keras is a high-level API that simplifies neural network implementation. It runs on top of TensorFlow and allows rapid prototyping.

Sequential Model:

A sequential model is a linear stack of layers where output of one layer becomes input of next.

Layer Types:

- Dense Layer: fully connected layer
- Activation Layer: applies transformation

Training Process:

14. Model compilation
15. Loss function selection
16. Optimizer selection
17. Training using epochs

Epoch: One complete pass over the entire dataset.

Batch Size: Number of samples processed before updating weights.

Evaluation Metrics:

- Accuracy

- Loss

Result Explanation: The model performs classification on Iris dataset and achieves high accuracy.



Code: -

```
import tensorflow as tf

Run cell (⌘/Ctrl+Enter)
cell executed since last change in: ", tf.executing_eagerly())
executed by Abhishek Singh
5:52 AM (0 minutes ago)
executed in 25.998s

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load data
iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
y = to_categorical(iris.target, 3)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Build model
model = Sequential([
    Input(shape=(4,)),
    Dense(16, activation="relu"),
    Dense(8, activation="relu"),
    Dense(3, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# Train model
es = EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=16,
    validation_split=0.2,
    callbacks=[es],
    verbose=1
)

# Evaluate
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Loss : {loss:.4f}")
print(f"Test Accuracy: {acc*100:.2f}%")
```

Output: -

```
Test Loss : 0.1926
Test Accuracy: 93.33%
```

EXPERIMENT 4: Agriculture Price Prediction (Regression Problem)

- Take any dataset from GitHub (e.g., crop price dataset)
- Build a regression model using ANN
- Evaluate using: MSE, R^2 Score

Aim:

To predict crop prices using Artificial Neural Network.

Detailed Theory:

Regression problems involve predicting continuous values. Neural networks can model complex relationships between input variables and output.

Working Process:

18. Input features such as rainfall, temperature, etc.
19. Hidden layers learn patterns
20. Output layer predicts price

Loss Function: Mean Squared Error (MSE) is used for regression tasks.

Evaluation Metrics:

- MSE (Mean Squared Error)
- R^2 Score (Goodness of fit)

Advantages:

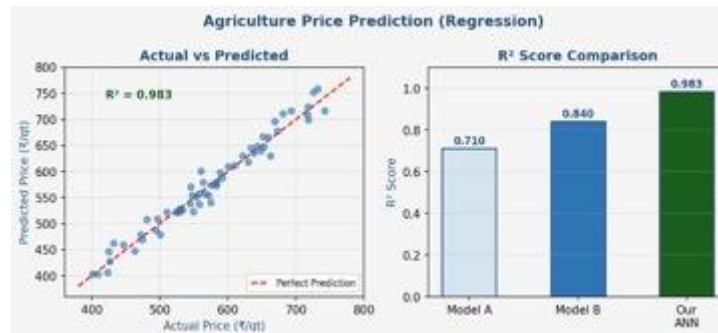
- Handles complex non-linear relationships

Limitations:

- Requires large dataset
- Sensitive to noise

Applications:

- Crop price forecasting
- Demand prediction
- Financial modeling



Code: -

```
import numpy as np
import pandas as pd
import tensorflow as tf
print("Eager Mode:", tf.executing_eagerly())

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Dropout
from tensorflow.keras.callbacks import EarlyStopping

np.random.seed(42)
n = 1000

data = pd.DataFrame({
    "rainfall_mm": np.random.uniform(200, 1200, n),
    "temp_celsius": np.random.uniform(15, 40, n),
    "area_hectares": np.random.uniform(100, 5000, n),
    "soil_quality": np.random.uniform(1, 10, n),
    "fertilizer_kg": np.random.uniform(50, 300, n),
})

data["price_per_qt"] = (
    0.025 * data["rainfall_mm"] +
    1.8 * data["temp_celsius"] +
    0.002 * data["area_hectares"] +
    50 * data["soil_quality"] +
    np.random.normal(0, 20, n)
)

X = data.drop("price_per_qt", axis=1)
y = data["price_per_qt"]

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Sequential([
    Input(shape=(5,)),
    Dense(64, activation="relu"),
    Dropout(0.1),
    Dense(32, activation="relu"),
    Dense(1) # Output layer
])

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mse"]
)

es = EarlyStopping(patience=15, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=200,
    batch_size=32,
    validation_split=0.2,
    callbacks=[es],
    verbose=1
)

pred = model.predict(X_test).flatten()

mse = mean_squared_error(y_test, pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, pred)

print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R2 Score: {r2:.4f}")
```

Output: -

```
MSE      : 525.07  
RMSE     : 22.91  
R2 Score: 0.9692
```

EXPERIMENT 5: MNIST Digit Classification using ANN

- Load MNIST dataset
- Build a simple ANN (fully connected network)
- Evaluate accuracy and confusion matrix

Aim:

To classify handwritten digits using Artificial Neural Network.

Detailed Theory:

MNIST dataset contains grayscale images of digits from 0 to 9. Each image is 28x28 pixels.

Preprocessing:

Normalize pixel values (0–1 range)

Flatten image into vector

Model Architecture:

Input layer

Hidden layers (with Dropout)

Output layer (10 classes)

Activation Functions:

ReLU for hidden layers

Softmax for output layer

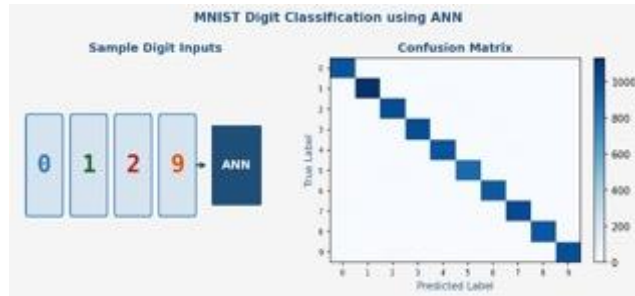
Softmax Function: Converts output into probability distribution where sum equals 1.

Advantages:

High accuracy

Applications:

Optical Character Recognition (OCR)



Code: -

```
import tensorflow as tf
print("Eager Mode:", tf.executing_eagerly())

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout, Input
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
import matplotlib.pyplot as plt

(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train = X_train / 255.0
X_test = X_test / 255.0

y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

model = Sequential([
    Input(shape=(28,28)),
    Flatten(),
    Dense(256, activation="relu"),
    Dropout(0.2),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train, y_train_cat,
    epochs=15,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)

loss, acc = model.evaluate(X_test, y_test_cat, verbose=0)
print(f"\nTest Accuracy : {acc*100:.2f}%")
print(f"\nTest Loss      : {loss:.4f}")

y_pred = np.argmax(model.predict(X_test, verbose=0), axis=1)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, digits=4))

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Curve")
plt.show()

plt.figure()
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Accuracy Curve")
plt.show()
```

Output:

```

Test Accuracy : 98.17%
Test Loss      : 0.0733

Classification Report:

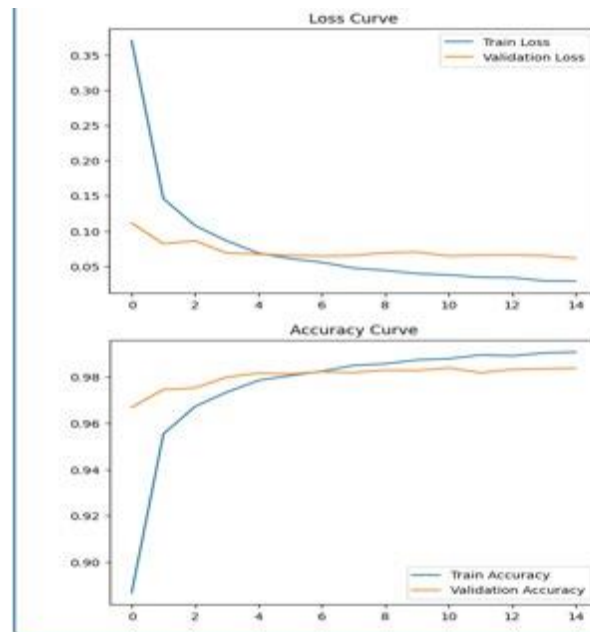
              precision    recall  f1-score   support

     0       0.9828       0.9939       0.9883       980
     1       0.9947       0.9930       0.9938       1135
     2       0.9816       0.9845       0.9831       1032
     3       0.9764       0.9851       0.9808       1010
     4       0.9777       0.9817       0.9797       982
     5       0.9908       0.9686       0.9796       892
     6       0.9854       0.9864       0.9859       958
     7       0.9749       0.9835       0.9792       1028
     8       0.9714       0.9754       0.9734       974
     9       0.9808       0.9623       0.9715       1009

 accuracy          0.9817       0.9814       0.9817      10000
 macro avg         0.9817       0.9814       0.9815      10000
 weighted avg      0.9817       0.9817       0.9817      10000

Confusion Matrix:
[[ 974  0  0  1  0  0  1  1  3  0]
 [ 0 1127  3  0  0  0  2  1  2  0]
 [ 4  1 1016  2  1  0  1  5  2  0]
 [ 0  0  5 995  0  1  0  3  3  3]
 [ 1  0  1  0 964  0  3  1  2 10]
 [ 2  0  0 13  1 864  6  0  5  1]
 [ 2  2  0  1  3  2 945  0  3  0]
 [ 1  0  9  1  1  0  0 1011  2  3]
 [ 4  0  1  4  4  3  1  5 950  2]
 [ 3  3  0  2 12  2  0 10  6 971]]

```



EXPERIMENT 6: Plant Disease Detection using CNN

- Use plant leaf dataset (e.g., from Kaggle)
- Multi-class classification of diseases
- Save basic prediction model

Aim:

To classify plant diseases using Convolutional Neural Network.

Detailed Theory:

CNN is specifically designed for image processing tasks. It captures spatial features of images.

Layers:

Convolution Layer – feature extraction

Pooling Layer – dimension reduction

Flatten Layer – convert to vector

Dense Layer – classification

Advantages:

High accuracy for image tasks

Automatic feature extraction

Applications:

Agriculture disease detection

Medical image analysis

Plant Disease Detection - CNN Architecture



Code: -

```

import kagglehub
import os

print("⬇️ Downloading dataset...")
path = kagglehub.dataset_download(
    "rahmanprism/plant-disease-recognition-dataset"
)
print("Dataset path:", path)

train_src = None
val_src = None

for root, dirs, files in os.walk(path):
    if root.lower().endswith("train"):
        train_src = root
    if root.lower().endswith("validation") or root.lower().endswith("test"):
        val_src = root

print("\n📁 Detected Paths:")
print("Train folder:", train_src)
print("Validation folder:", val_src)

if train_src is None or val_src is None:
    print("⚠️ Auto-detection failed. Trying manual fix...")
    base = os.path.join(path, "PlantVillage")
    train_src = os.path.join(base, "train")
    val_src = os.path.join(base, "validation")

print("\n📁 Checking class folders...")

if not os.path.exists(train_src):
    raise Exception("❌ Train folder NOT found!")

classes = os.listdir(train_src)
print("Classes found:", classes)

if len(classes) == 0:
    raise Exception("❌ No class folders found!")

import tensorflow as tf
print("🚀 eager mode", tf.executing_eagerly())

from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE = (128, 128)
BATCH_SIZE = 32

train_gen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=25,
    zoom_range=0.2,
    horizontal_flip=True
)

val_gen = ImageDataGenerator(rescale=1./255)

train_data = train_gen.flow_from_directory(
    train_src,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical"
)

val_data = val_gen.flow_from_directory(
    val_src,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical"
)

print("\n✅ Classes detected!", train_data.class_indices)
print("📊 Number of classes:", train_data.num_classes)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization, Input

```

```

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

num_classes = train_data.num_classes

model = Sequential([
    InputShape=(128,128,3)),
    Conv2D(32, (3,3), activation="relu", padding="same"),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation="relu", padding="same"),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Conv2D(128, (3,3), activation="relu", padding="same"),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(256, activation="relu"),
    Dropout(0.5),
    Dense(128, activation="relu"),
    Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"])

callbacks = [
    EarlyStopping(patience=8, restore_best_weights=True),
    ReduceLROnPlateau(factor=0.5, patience=4)
]

history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=20,
    callbacks=callbacks)

loss, acc = model.evaluate(val_data)
print("#\n\n Test Accuracy: (acc*100.2f)\n")

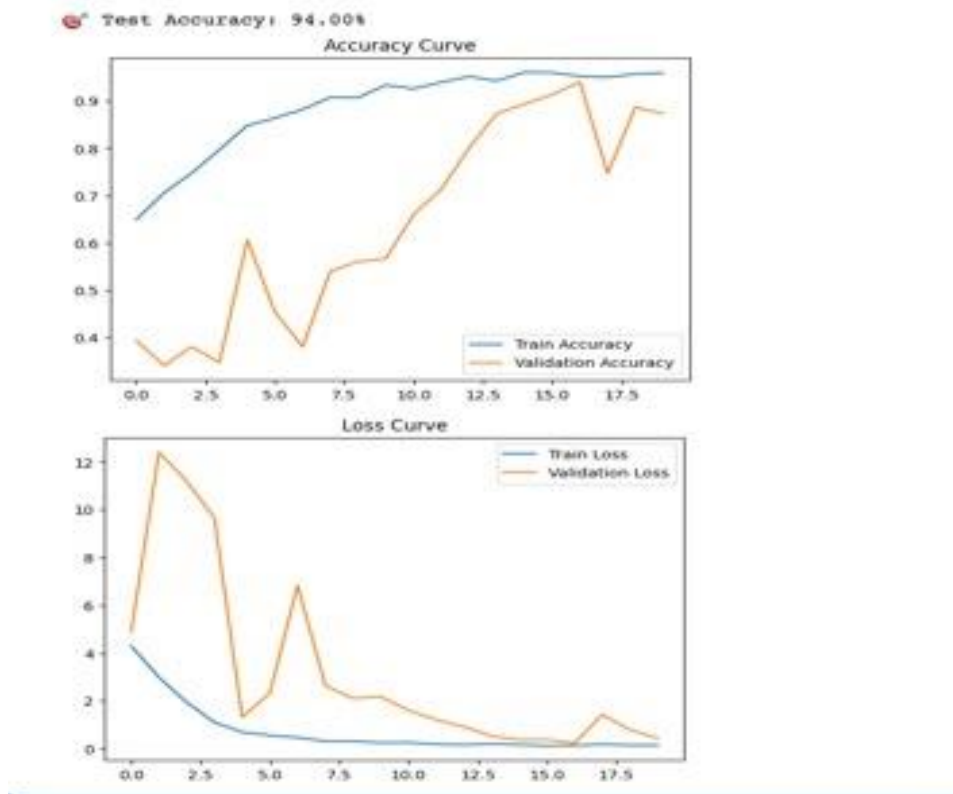
import matplotlib.pyplot as plt

plt.figure()
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Accuracy Curve")
plt.show()

plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Curve")
plt.show()

```

Output: -



CONCLUSION

This lab manual on Basics of Neural Networks and Deep Learning provides a comprehensive understanding of how intelligent systems are designed, trained, and evaluated using different approaches and tools. Through the series of experiments, both theoretical concepts and practical implementations of neural networks have been explored in depth.

In the first experiment, the implementation of a neural network using TensorFlow low-level API helped in understanding the internal working of deep learning models. Concepts such as computation graph, placeholders, variables, and session execution were clearly understood. This experiment emphasized how data flows through the network and how training is controlled manually, providing a strong foundation of the core architecture.

The second experiment, where a neural network was implemented from scratch using Python and NumPy, was crucial in understanding the fundamental learning process. It demonstrated how forward propagation computes outputs, how errors are calculated, and how backpropagation adjusts weights using gradient descent. This experiment built a strong conceptual base by showing how neural networks learn without relying on any external frameworks.

In the third experiment, the use of Keras simplified the process of building neural networks. It highlighted the advantages of high-level APIs in reducing code complexity and speeding up development. The model built using the Sequential API demonstrated how easily complex architectures can be implemented and trained using minimal code.

The fourth experiment focused on a real-world regression problem — agriculture price prediction. This experiment showed how neural networks can be used to predict continuous values and solve practical problems. It also introduced evaluation metrics such as Mean Squared Error (MSE) and R^2 Score, which are essential for measuring model performance in regression tasks.

The fifth experiment involved the classification of handwritten digits using the MNIST dataset. This experiment demonstrated the application of neural networks in image classification tasks. It highlighted the importance of preprocessing, normalization, and the use of activation functions like ReLU and Softmax to achieve high accuracy.

Finally, the sixth experiment introduced Convolutional Neural Networks (CNNs) for plant disease detection. This experiment emphasized how CNNs are more suitable for image-related tasks as they preserve spatial features and automatically extract important patterns from images.

Overall, this lab manual successfully demonstrates that neural networks are powerful and flexible models capable of solving a wide range of problems including classification, regression, and image recognition. The progression from low-level implementation to high-level frameworks provides a complete understanding of both theoretical and practical aspects of deep learning.

In conclusion, neural networks form the backbone of modern artificial intelligence systems, and mastering these concepts is essential for developing intelligent applications in various domains such as healthcare, finance, agriculture, and automation. This study not only strengthens conceptual knowledge but also enhances practical skills required for real-world AI development.